

METODOLOGÍA I

Trabajo Práctico — Mi biblioteca de prompts

Ficha 1 - Anatomía de 4 componentes

Situación de uso: Generar una función específica para conectarse a una base de datos MySQL y realizar una consulta, sin que el LLM agregue código basura, explicaciones innecesarias o use librerías equivocadas.

Patrón aplicado: Anatomía de 4 componentes. Se elige porque delimita exactamente qué se necesita, bajo qué contexto, qué no debe hacer y cómo debe entregarlo, evitando idas y vueltas.

Prompt completo:

Objetivo: Escribe una función en Python que se conecte a una base de datos MySQL y devuelva los resultados de una consulta SELECT.

Contexto: Estoy desarrollando el backend básico de una aplicación y necesito la conexión pura usando mysql-connector-python.

Restricciones: Usa bloques try-except para manejar los errores de conexión. No uses ORMs como SQLAlchemy.

Formato de salida: Devuelve únicamente el bloque de código, sin explicaciones adicionales ni formato markdown.

Resultado/observación: Frente a un pedido vago como "hacé una conexión a mysql", esta versión evitó que el modelo diera un tutorial completo de bases de datos o instalara librerías no deseadas, entregando un código listo para implementar.

Ficha 2 - Few-Shot Prompting

Situación de uso: Extraer información estructurada (acción y tabla) a partir de comandos en lenguaje natural dados por un usuario.

Patrón aplicado: Few-Shot Prompting. Es ideal para tareas de extracción o clasificación porque los ejemplos muestran el patrón exacto de formato esperado y le enseñan al modelo qué hacer con casos no válidos.

Prompt completo:

Extrae la acción SQL (INSERT, UPDATE, SELECT, DELETE) y el nombre de la tabla a partir de la frase del usuario. Sigue estrictamente este formato:

Frase: 'Agrega un nuevo cliente con nombre Juan a la base.' -> Acción: INSERT | Tabla: clientes

Frase: 'Buscame todos los productos que cuesten más de \$1000.' -> Acción: SELECT | Tabla: productos

Frase: 'El sistema es muy lento hoy y la pantalla parpadea.' -> Acción: NINGUNA | Tabla: N/A

Frase: 'Borrá el registro del empleado con ID 45.' ->

Resultado/observación: Al incluir ejemplos (especialmente el negativo donde la frase no tiene relación con bases de datos), la respuesta se estandarizó en un formato que un script puede procesar fácilmente, en lugar de recibir respuestas conversacionales del LLM.

Ficha 3 - Role Prompting

Situación de uso: Revisar un snippet de código en Java en busca de vulnerabilidades y malas prácticas.

Patrón aplicado: Role Prompting. Al forzar al modelo a asumir un rol senior o especializado, se eleva el rigor técnico de la respuesta y se priorizan aspectos de arquitectura y seguridad.

Prompt completo:

Sin rol: Revisá este código Java y decime si está bien: `public void process(String input) { System.out.println(input); }`

Con rol: Actúa como un Desarrollador Senior y experto en ciberseguridad especializado en Java 21. Revisa el siguiente fragmento de código enfocándote en posibles vulnerabilidades de inyección, fugas de memoria y adherencia a las convenciones de Clean Code. Código: `public void process(String input) { System.out.println(input); }`

Resultado/observación: Sin el rol, el modelo se limitó a decir que la sintaxis era correcta. Con el rol asignado, el modelo identificó riesgos de Log Forging (falsificación de registros por no sanitizar el input) y recomendó usar un framework de logging estándar como SLF4J en lugar de imprimir directamente en consola.

Ficha 4 - Prompt Estructurado

Situación de uso: Refactorizar y optimizar un bucle en un archivo de Python, asegurando que las funciones matemáticas de la parte superior del archivo queden intactas.

Patrón aplicado: Prompt estructurado. El uso de delimitadores es fundamental cuando se trabaja con código heredado o archivos largos donde solo queremos que la IA opere sobre una porción muy específica.

Prompt completo:

Refactoriza el código ubicado entre las etiquetas para usar comprensiones de listas y hacerlo más eficiente. NO modifiques bajo ninguna circunstancia el código ubicado entre las etiquetas <NO_TOCAR>.

<NO_TOCAR>

```
import math
```

```
def calcular_distancia(p1, p2):
```

```
    return math.sqrt((p2[0]-p1[0])**2 + (p2[1]-p1[1])**2)
```

</NO_TOCAR>

Resultado/observación: Un prompt ingenuo corría el riesgo de que el LLM reescribiera todo el archivo e importara librerías diferentes (como numpy). Con delimitadores, el modelo se limitó a devolver la comprensión de lista de la sección marcada, respetando la estructura global.

Ficha 5 - Chain of Thought

Situación de uso: Diseñar la lógica matemática para un sistema de paginación de resultados, donde un mínimo error en los índices rompe el resultado final.

Patrón aplicado: Chain of Thought. Obliga al modelo a explicitar la matemática detrás de los índices antes de escupir código, lo que reduce drásticamente las alucinaciones lógicas.

Prompt completo:

Directo: Escribí una función que calcule los índices de inicio y fin para paginar una lista, recibiendo la página actual y la cantidad de items por página.

Paso a paso (CoT): Dada la página actual (entero, empezando en 1) y la cantidad de ítems por página (entero). Primero, razona paso a paso cómo calcular el índice de inicio,

recordando que las listas empiezan en 0. Segundo, explica matemáticamente cómo deducir el índice final. Tercero, escribe la función resultante en Python.

Resultado/observación: El prompt directo a veces fallaba con el índice base cero (generaba inicio = pagina * items). Al pedir el razonamiento paso a paso, el modelo explicó que para la página 1 el índice debe ser 0, llegando a la fórmula correcta (pagina - 1) * items y traduciéndola luego a código funcional sin errores lógicos.

Ficha 6 - Ingeniería Iterativa

Situación de uso: Crear una clase base en Python para la generación procedural de mapas divididos en "chunks" de 10x10.

Patrón aplicado: Ingeniería iterativa. Ningún prompt complejo sale bien a la primera; se necesita converger refinando las instrucciones tras analizar lo que devuelve el modelo.

Prompt completo:

Iteración 1: Escribe un código en Python para generar mapas por chunks.

(El modelo devolvió un script monolítico usando la librería pygame para dibujar mapas, cosa que no se había pedido).

Iteración 2: Crea una clase en Python llamada GeneradorMapa que maneje la creación de chunks. Cada chunk debe ser de 10x10. Debe incluir un método generar_chunk(x, y) que devuelva una matriz de 10x10 con valores aleatorios enteros entre 0 y 1 (representando tipos básicos de terreno). No uses librerías gráficas, solo lógica pura orientada a objetos.

Resultado/observación: En la primera iteración se obtuvo código difícil de integrar por estar acoplado a una interfaz gráfica. En la segunda vuelta, al especificar las dimensiones del chunk (10x10) y restringir el uso de librerías visuales, el resultado fue una clase limpia, testeable y lista para usarse como base algorítmica del proyecto.